

TERESA — Braccio Unitree Z1 + WBC Spot

Diario di sviluppo, TODO list e guida di avvio

Branch attivo: **TERESA** | Repo: TERESA_ws/src/

Ultimo aggiornamento: **17 Aprile 2026**

1 Panoramica del sistema

Il sistema controlla il braccio **Unitree Z1** montato su **Boston Dynamics Spot** per eseguire autonomamente il protocollo **FAST ultrasound** su un paziente sdraiato. Lo stack completo è composto da:

- **Percezione Spot (Orbbec)**: YOLO11 skeleton + classificazione postura + rilevamento persona sdraiata + approach point (`spot_perception`)
- **Percezione Z1 (RealSense)**: YOLO11 torso tracker + Kalman 3D + stima piano superficiale PCA (`z1_vision`)
- **WBC — Whole Body Control** (`spot_control`): QP olistico Spot+Z1 durante l'approccio; FSM 6 stati; mux priorità arm goal
- **Controllo posizione Z1**: IK Jacobiana pseudo-inversa + JointTrajectoryController
- **Controllo forza Z1**: Impedance controller cartesiano (500 Hz, torque control)
- **Sicurezza**: keyboard safety, stati HOMING/EMERGENCY, workspace clipping

Flusso della missione

1. **Approaching**: WBC QP coordina Spot (`cmd_vel`) + Z1 (`ik_goal`) simultaneamente; braccio pre-punta il paziente mentre Spot cammina
2. **Handoff**: Orbbec cede a RealSense; `z1_FSM` prende controllo del braccio
3. **FAST scan**: `z1_FSM` esegue 5 punti anatomici (Hub, Subxiphoid, RUQ, LUQ, Suprapubic) con impedance control; Spot fermo
4. **WS Extension**: se punto FAST irraggiungibile, WBC torna attivo per micro-step Spot + compensazione braccio simultanea

2 Branch attivi

Branch TERESA — branch principale attivo (2026-04-17)

Contiene l'intero stack integrato: FAST protocol, body search scan 3 fasi, impedance controller, `spot_perception`, `spot_control` (WBC).

3 Come avviare il sistema

Prerequisiti

- SpotCore con `spot_ros2` driver in esecuzione
- Jetson con ROS2 Humble, Pinocchio, driver Z1, RealSense SDK, Orbbec SDK, YOLO11
- `ROS_DOMAIN_ID` uguale su Jetson e SpotCore

- Workspace compilato: `colcon build -packages-select z1_vision spot_perception spot_control`

Sequenza di lancio completa (sistema integrato)

```
# Su ogni terminale Jetson:
source /opt/ros/humble/setup.bash && source ~/TERESA_ws/install/setup.
bash
export ROS_DOMAIN_ID=0
```

T	Macchina	Comando
1	SpotCore	<code>ros2 launch spot_ros2 spot_driver.launch.py</code>
2	Jetson	<code>ros2 launch spot_perception spot_perception.launch.py</code>
3	Jetson	<code>ros2 launch z1_vision z1_realsense.launch.py use_rviz:=true</code>
4	Jetson	<code>ros2 launch z1_vision z1_perception.launch.py</code>
5	Jetson	<code>ros2 launch z1_vision z1_control.launch.py fsm_delay:=5.0</code>
6	Jetson	<code>ros2 launch spot_control wbc.launch.py</code>
7	Jetson	<code>ros2 run z1_vision z1_keyboard_safety</code>

Nota: il terminale 6 (WBC) va avviato dopo T1–T5. Il `wbc_coordinator` parte in IDLE e inizia l’approccio solo quando `spot_perception` rileva una persona sdraiata con confidenza sufficiente.

Parametri utili

- `fsm_delay:=5.0` — secondi prima che `z1_FSM` inizi (lascia tempo al setup)
- `use_rviz:=true` — lancia RViz2 con la configurazione del sistema
- `z1_x:=0.30 z1_z:=0.70` — offset TF `body→link00` (stimato; misurare dopo montaggio fisico)

Comandi tastiera (`z1_keyboard_safety`)

Tasto	Azione
H	Invia braccio alla home position (qualunque stato)
ESC	Emergenza: stop immediato, poi HOMING
R	Reset dallo stato EMERGENCY
Q	Chiude il nodo keyboard safety

4 Architettura

FSM z1_FSM.py — stati

Stato	Descrizione
HOMING	Stato iniziale. Porta il braccio in home pose via IK/JTC
WAITING	Aspetta target LOCKED da RealSense tracker
BODY_SCANNING	Scansione a griglia per trovare posizione ottimale del torso
CHECKING_WORKSPACE	Verifica raggiungibilità target (Pinocchio, non bloccante)
APPROACHING	Invia goal IK verso il target
WAIT_IK_DONE	Attende /ik_done=True dal nodo IK
SWITCHING_TO_TORQUE	Switch JTC → torque_controller
IMPEDANCE_RUNNING	Impedance controller: approach → hold → retract
SWITCHING_TO_JTC	Switch torque_controller → JTC
SCAN_PRELIFT	Retrazione tra punti FAST
EMERGENCY	Stop bloccante, attende reset da tastiera
FAULT	Errore fatale (switch fallito)

FSM wbc_coordinator.py — stati

Stato	Descrizione
IDLE	Attende rilevamento LYING con confidenza $\geq$ soglia
APPROACHING	QP attivo: Spot naviga + Z1 punta il paziente simultaneamente
CONFIRMING	Confidenza Orbbec scesa: braccio mantiene ultima stima
HANDOFF	Orbbec perso: WBC disabilitato, z1_FSM prende controllo
SCANNING	z1_FSM attivo, WBC dormiente, Spot fermo
WS_EXTENSION	Punto FAST irraggiungibile: QP micro-step Spot + arm compensation

Topic principali

Topic	Tipo	Descrizione
/z1/ik_goal_pose	PoseStamped	Goal IK da z1_FSM (entra nel mux)
/wbc/ik_goal_pose	PoseStamped	Goal IK da WBC QP (entra nel mux)
/ik_goal_pose	PoseStamped	Uscita mux → z1_ik_to_jtc
/wbc/enable	Bool	Attiva priorità WBC nel mux
/ik_enable	Bool	Abilita nodo IK
/ik_done	Bool	Fine movimento JTC
/my_spot/cmd_vel	Twist	Velocità base Spot (da WBC QP)
/wbc/ee_goal	PoseStamped	Goal EE da wbc_coordinator a QP
/wbc/state	String	Stato FSM coordinator
/wbc/ws_request	Bool	z1_FSM chiede WS extension
/z1_fsm/state	String	Stato FSM z1
/torso_target_ee_locked	PoseStamped	Target EE quando tracker LOCKED
/torso_surface_frame	PoseStamped	Piano superficie torso (PCA)
/surface_signed_distance	Float32	Distanza TCP-superficie
/laying_human/approach_point	PoseStamped	Approach point da Orbbe
/human_pose/posture	String	STANDING / SITTING / LYING
/human_pose/posture_confidence	Float32	Confidenza classificatore postura

5 DONE — Funzionalità implementate

Stack Z1 (z1_vision)

- ✓ Pipeline base: YOLO11 + Kalman 3D + FSM IDLE/ESTIMATING/LOCKED
- ✓ IK Jacobiana pseudo-inversa smorzata + smoothstep quintico + JTC
- ✓ `realsense_surface_node`: stima piano torso da depth ROI (PCA)
- ✓ Impedance controller (500 Hz): APPROACH → HOLD → RETRACT con compensazione gravity+coriolis
- ✓ `safe_controller_switch`: switch JTC ↔ torque
- ✓ Workspace checker (Pinocchio + L-BFGS-B), clip a `max_reach - 30 cm`
- ✓ HOMING, EMERGENCY, keyboard safety (H/ESC/R/Q)
- ✓ Body Search Scan 3 fasi: wrist sweep → arco → refinement adattivo; early stop, fallback, stima 3D multi-vista
- ✓ Protocollo FAST: Hub → Subxiphoid → RUQ → LUQ → Suprapubic; punti calcolati da keypoint YOLO, clippati a bbox torso
- ✓ Test su robot reale: ciclo FAST completo funzionante

Stack Spot (spot_perception)

- ✓ Package `spot_perception`: YOLO11 skeleton, posture classifier, laying human detector, approach point
- ✓ Launch file con parametri GPU Jetson (`device: '0', use_half: true`)

- ✓ Driver Orbbec (`src/orbbec_camera`) presente in workspace

WBC Spot+Z1 (`spot_control`) — aprile 2026

- ✓ Refactoring: `teresa_coordinator` + `spot_navigation` unificati in `spot_control`
- ✓ `teresa_mission.py`: geometria approccio laterale (asse corpo COCO, scelta lato, goal locking, crouch); ora usato solo per calcolo geometria, navigazione gestita da WBC
- ✓ `wbc_math.py`: funzioni pure QP — `compute_j_base`, `compute_j_holistic`, `manipulability`, `wbc_split` (weighted damped least-squares); 14/14 test pytest
- ✓ `wbc_qp_controller.py`: nodo ROS2 — legge TF + `joint_states`, Jacobiano olistico (`J_arm` Pinocchio + `J_base` ruotato in odom), QP ogni 1.5 s, pubblica `/wbc/ik_goal_pose` + `/my_spot/cmd_vel`
- ✓ `wbc_coordinator.py`: FSM 6 stati — gestisce fasi approccio, handoff Orbbec→RealSense, scanning, workspace extension
- ✓ `ik_goal_mux.py`: multiplexer con priorità — WBC > z1_FSM su `/ik_goal_pose` quando `/wbc/enable=True`
- ✓ `wbc.launch.py`: TF statico `body`→`link00` (parametrizzabile) + mux + QP controller + coordinator

6 TODO

Alta priorità — Jetson setup & integrazione

- ☐ **Jetson setup**: ROS2 Humble, Pinocchio, driver Z1, RealSense SDK, Orbbec SDK, YOLO11
- ☐ **Test spot_perception su Jetson**: verificare `yolo_skeleton`, `posture_classifier`, `laying_human_detector` con Orbbec reale
- ☐ **Configurare ROS_DOMAIN_ID** comune Jetson ↔ SpotCore; verificare che i topic critici attraversino la rete

Alta priorità — calibrazione hardware

- ☐ **Misurare offset fisico** `body`→`link00`: dopo montaggio Z1 su Spot, aggiornare `wbc.launch.py` (`z1_x`, `z1_y`, `z1_z`)
- ☐ **Verificare Y clipping FAST**: controllare che tutti i punti rimangano entro il bounding box del torso nelle varie posture paziente

Alta priorità — test WBC

- ☐ **Test APPROACHING**: verificare che braccio si muova simultaneamente a Spot; monitorare log `wbc_qp_controller` (`m`, `vx`, `wz`, `|dp|`)
- ☐ **Tuning QP**: regolare `lam_arm`, `lam_base`, `kp_pos` in `wbc_params.yaml` dopo primi test reali

Media priorità

- ☐ **Unificare pipeline YOLO sulla Jetson:** singola istanza YOLO11 per Orbbec (skeleton 17 joint) e RealSense (torso center), evitando doppio caricamento pesi in VRAM
- ☐ **Tuning griglia body scan:** valutare se `ext_y` e `ext_z` coprono correttamente la zona torso sul robot reale

Bassa priorità / futuri miglioramenti

- ☐ **Logging:** ROS bag recording automatico al launch con i topic principali (FSM state, torso target, scan point, impedance)
- ☐ **Visualizzazione RViz:** marker per WBC state, approach point, workspace reachability, skeleton Spot
- ☐ **Multi-persona:** gestione del caso in cui YOLO rileva più persone (attualmente prende la prima/più vicina)

7 Sequenza di avvio dettagliata

Prerequisito: workspace compilato e sourced su Jetson.

```
cd ~/TERESA_ws
colcon build --packages-select z1_vision spot_perception spot_control
source install/setup.bash
export ROS_DOMAIN_ID=0    # deve matchare su Jetson e SpotCore
```

Livello 0 — Solo percezione Spot (Orbbec, Spot spento)

Test rilevamento scheletro, postura, approach point con Orbbec reale, senza Spot e senza Z1.

```
# Terminale 1      Jetson
ros2 launch spot_perception spot_perception.launch.py

# Terminale 2      monitoraggio
ros2 topic echo /human_pose/posture
ros2 topic echo /human_pose/posture_confidence
ros2 topic echo /laying_human/approach_point
```

RViz — topic utili: `/human_pose/skeleton_markers`, `/human_pose/bounding_box`, `/laying_human/bbox`

Livello 1 — Solo Z1 (senza Spot)

Test braccio Z1 standalone: homing, body scan, FAST scan con RealSense.

```
# Terminale 1      hardware + TF
ros2 launch z1_vision z1_realsense.launch.py use_rviz:=true

# Terminale 2      percezione RealSense
ros2 launch z1_vision z1_perception.launch.py

# Terminale 3      FSM + IK + impedance
ros2 launch z1_vision z1_control.launch.py fsm_delay:=5.0

# Terminale 4      safety
ros2 run z1_vision z1_keyboard_safety
```

Livello 2 — Percezione + geometria approccio (Spot spento)

Verifica la geometria di approccio in RViz senza muovere Spot. `teresa_mission` calcola asse corpo, goal laterale, safe zone.

```
# Terminale 1      percezione Spot
ros2 launch spot_perception spot_perception.launch.py

# Terminale 2      geometria approccio (dry_run = default true, Spot non
                    si muove)
ros2 launch spot_control teresa_mission.launch.py
```

RViz — topic coordinator: `/teresa/body_axis`, `/teresa/approach_goal`, `/teresa/safe_zone`, `/teresa/fsm_state`

Livello 3 — Sistema completo TERESA (Spot + Z1 + WBC)

Sistema integrato: WBC coordina approccio Spot+Z1, poi `z1_FSM` esegue FAST scan.

```
# Terminale 1      SpotCore (SSH)
ros2 launch spot_ros2 spot_driver.launch.py

# Terminale 2      Jetson: percezione Spot (Orbbec)
ros2 launch spot_perception spot_perception.launch.py

# Terminale 3      Jetson: hardware Z1 + RealSense + TF
ros2 launch z1_vision z1_realsense.launch.py use_rviz:=true

# Terminale 4      Jetson: percezione Z1 (YOLO torso + surface)
ros2 launch z1_vision z1_perception.launch.py

# Terminale 5      Jetson: FSM + IK + impedance
ros2 launch z1_vision z1_control.launch.py fsm_delay:=5.0

# Terminale 6      Jetson: WBC (mux + QP controller + coordinator)
ros2 launch spot_control wbc.launch.py
# dopo montaggio fisico Z1:
# ros2 launch spot_control wbc.launch.py z1_x:=0.28 z1_z:=0.65

# Terminale 7      Jetson: safety
ros2 run z1_vision z1_keyboard_safety
```

Monitoraggio:

```
ros2 topic echo /wbc/state          # fase WBC corrente
ros2 topic echo /z1_fsm/state       # fase Z1 corrente
ros2 topic echo /human_pose/posture # rilevamento paziente
ros2 topic echo /laying_human/approach_point # goal approccio
```

8 File principali

File	Ruolo
<i>Package <code>z1_vision</code> (Jetson)</i>	
<code>z1_FSM.py</code>	FSM principale Z1 (12 stati, FAST protocol)

File	Ruolo
body_search_scanner.py	Scan a griglia 3 fasi per trovare torso
z1_ik_to_jtc.py	IK Pinocchio → JTC action
impedance_controller_realsense.py	Impedenza cartesiana 500 Hz, torque control
safe_controller_switch.py	Switch JTC ↔ torque controller
workspace_checker.py	Raggiungibilità workspace (Pinocchio + scipy)
z1_yolo_torso_tracker.py	YOLO11 + Kalman 3D + lock + scan mode
realsense_surface_node.py	Stima piano torso (PCA depth ROI)
z1_keyboard_safety.py	Safety node interattivo
<i>Package spot_perception (Jetson, Orbbec)</i>	
yolo_skeleton_spot.py	YOLO11 skeleton 17 joint + Kalman + TF
posture_classifier.py	Classifica STANDING / SITTING / LYING
laying_human_detector.py	Persona sdraiata → approach point
depth_processing.py	Depth lookup con window mediana + outlier filter
<i>Package spot_control (Jetson)</i>	
wbc_math.py	Funzioni pure QP: J_base, J_holistic, manipulability, wbc_split
wbc_qp_controller.py	Nodo QP: TF+joints → WBC split → /wbc/ik_goal_pose + cmd_vel
wbc_coordinator.py	FSM 6 stati: IDLE/APPROACHING/-CONFIRMING/-HANDOFF/SCANNING/-WS_EXTENSION
ik_goal_mux.py	Mux priorità: WBC > z1_FSM su /ik_goal_pose
teresa_mission.py	Geometria approccio laterale Spot + crouch
<i>Config files</i>	
z1_fsm_params.yaml	Parametri FSM Z1 (home, topic, workspace)
z1_ik_jtc_params.yaml	IK/JTC (ik_rot_weight: 0.8)
z1_yolo_torso_params.yaml	YOLO/Kalman/lock
impedance_control_params.yaml	Guadagni impedance, timing
body_search_params.yaml	Griglia scan (ny×nz, lookat, early stop)
wbc_params.yaml	QP gains, update rate, TF frames, velocity limits
<i>Launch files</i>	
z1_vision/z1_control.launch.py	FSM + IK + impedance + safety
z1_vision/z1_perception.launch.py	YOLO torso + surface node
z1_vision/z1_realsense.launch.py	HW Z1 + RealSense + TF
spot_control/wbc.launch.py	TF body→link00 + mux + QP + coordinator
spot_control/teresa_mission.launch.py	Geometria approccio (dry_run default)

9 Note tecniche

- **IK:** Jacobiana pseudo-inversa smorzata, frame LOCAL_WORLD_ALIGNED, smooth-step quintico (zero velocità a inizio/fine), joint unwrap con `_make_target_near`. `ik_rot_weight: 0.8` per EE perpendicolare alla superficie

- **WBC QP**: weighted damped least-squares, $w_{arm} = \lambda_{arm}/(m + \varepsilon)$, $w_{base} = \lambda_{base}$; alta manipolabilità $\Rightarrow$ braccio preferito, bassa $\Rightarrow$ base contribuisce di più. Update ogni 1.5s (rispetta `traj_min_time=1.0s` di `z1_ik_to_jtc`)
- **IK Goal Mux**: `z1_FSM` pubblica su `/z1/ik_goal_pose`; WBC pubblica su `/wbc/ik_goal_pose`; il mux seleziona in base a `/wbc/enable`
- **TF body→link00**: offset stimato $x = 0.30\text{ m}$, $z = 0.70\text{ m}$. **Da misurare fisicamente dopo montaggio Z1 su Spot**
- **URDF path** (hardcoded in yaml):
`/home/andrea/Ros2_repositories/unitree_z1_ws/install/
z1_description/share/z1_description/urdf/z1.urdf`
- **Kalman**: dt adattivo dal timestamp immagine, `vel_damping = 0.9`
- **Impedance**: 500 Hz, PID cartesiano + joint-space polso (J4–J6), compensazione gravità+coriolis